

Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 9

Rappresentazione dei caratteri

I caratteri di un testo vengono tradotti mediante sequenze di bit tramite un codice di traduzione.

Ciascun carattere occupa nel calcolatore 1 byte

Il più usato è il **codice ASCII** (American Standard Code for Information Interchange) a 7 bit quindi un massimo di 128 caratteri.

- Caratteri di comando: la loro scrittura su una periferica non determina una stampa ma una esecuzione di un comando. Per esempio “andare a capo”.
- Caratteri alfanumerici: maiuscole da A a Z, minuscole da a a z, cifre da 0 a 9. I simboli : operatori aritmetici e punteggiatura (Mancano dei caratteri, per esempio le lettere accentate e le lettere greche).

Varianti dell'ASCII: usano l'ottavo bit per rappresentare altri 128 caratteri. (UTF-8, UNICODE)

Caratteri

Le variabili di tipo carattere assumono valori alfanumerici: lettere dell'alfabeto, cifre decimali, punteggiatura etc. Per dichiararle

```
char x, y, z;
```

La definizione riserva uno spazio sufficiente per un carattere. Per assegnargli un valore:

```
x = 'A';  
y = ';';  
z = '&';
```

Per assegnare un valore costante a una variabile `char` lo si racchiude tra apici singoli

Caratteri

Carattere	Decimale	Esadecimale	Binario
A	65	41	01000001
;	59	3B	00111011
&	38	26	00100110

A ogni carattere corrisponde una rappresentazione numerica univoca.

Si possono confrontare due simboli con tutti gli operatori relazionali.

Nel codice ASCII le lettere alfabetiche maiuscole sono in ordine crescente da A (65) a Z (90), le minuscole vanno da a (98) a z (122) e le cifre decimali da 0 (48) a 9 (57).

Caratteri

Per visualizzare dei **char**:

```
printf("%c %c %c", x, y, z);
```

Per immettere caratteri:

```
scanf ("%c", &x)
```

In alternativa:

```
x = getchar(); /* immette un carattere */  
putchar(x);    /* visualizza un carattere */
```

ATTENZIONE: Il tasto di Invio corrisponde a un carattere. Nel caso di letture successive per intercettarlo è necessaria un'ulteriore lettura:

```
scanf ("%c", &mem) o mem=getchar ()
```

Caratteri

```
#include <stdio.h>
int main(void)
{
    char x, y, z;
    printf("Digita tre carat.: ");
    scanf("%c%c%c", &x, &y, &z);

    printf("Hai digitato: ");
    printf("%c%c%c\n", x, y, z);
}
```

```
#include <stdio.h>
int main(void)
{
    char x, y, z;
    printf("Digita tre carat.: ");
    x = getchar();
    y = getchar();
    z = getchar();
    printf("Hai digitato: ");
    putchar(x);
    putchar(y);
    putchar(z);
    putchar('\n');
}
```

Confronto fra char

Le istruzioni condizionali sono applicate anche ai caratteri;

```
char x;  
if(x=='A') printf("Si tratta di una A");
```

Il valore di `x` è un carattere il cui codice è maggiore o uguale del codice di `A`?

```
if(x>='A') printf("Maggiore o uguale di A");
```

Sono ammesse espressioni con valori interi e valori carattere:

```
if(x>='A' && i>100)  
    printf("x maggiore o uguale di A e i maggiore di  
100");
```

Confronto fra char

```
if(x>='A' && x<='Z')  
    printf("Si tratta di una lettera maiuscola");
```

L'espressione `x>='A'` è vera se il valore di `x` è maggiore o uguale ad `'A'`, l'espressione `x<='Z'` è vera se `x` è minore o uguale a `'Z'`; l'essere entrambe contemporaneamente vere permette di determinare dunque che `x` è una lettera maiuscola: A,B,C,D...Z.

Per verificare se il carattere è una lettera scriviamo:

```
if((x>='A' && x<='Z') || (x>='a' && x<='z'))  
    printf("Si tratta di una lettera");  
else  
    printf("Non si tratta di una lettera");
```

Iniziamo un nuovo punto di vista e
introduciamo:

- Funzioni in C
- Variabili locali

Sottoprogrammi C

In C i sottoprogrammi sono detti **funzioni**: a partire da uno o più valori in ingresso *ritornano* (o *restituiscono*) un valore al chiamante.



Un esempio di funzione è `abs`. Considerando la funzione come una **scatola nera**, tutto quello che dobbiamo sapere è che **immettendo** come argomento `i` ne **ritorna** il valore assoluto.



Vantaggi

- **Sviluppo più semplificato del programma.**
Il programma risulta anche più facile da leggere, interpretare, correggere o modificare.
- **Riusabilità del software:** blocchi di istruzioni che possono essere riutilizzati in più punti del programma. Con le funzioni si evita di ripetere codice.
- **Uso di funzioni esistenti** (librerie).
- **Astrazione:** si nascondono i dettagli interni (per esempio con le funzioni di libreria) perché si usa la funzione senza sapere in dettaglio come funziona ma si conosce soltanto l'input e l'output.

Usare le funzioni in C

Per poter essere utilizzata, una funzione deve essere:

- **dichiarata** (prima del **main**) fornendo il cosiddetto **prototipo di funzione**;
- **invocata** in un qualsiasi punto del programma in cui è utile.
- **definita** (dopo il **main**) come successione di istruzioni.
- ATTENZIONE: In C non si possono definire funzioni dentro funzioni.

Sottoprogrammi C

```
#include <stdio.h>
double cubo(float);
int main(void)
{
    float a;
    double b;
    printf("Inserisci un numero: ");
    scanf("%f", &a);
    b = cubo(a);
    printf("%f elevato al cubo è
    uguale a %f", a, b);
}

double cubo(float c)
{
    return (c*c*c);
}
```

dichiarazione:

```
double cubo(float);
```

invocazione:

```
b = cubo(a);
```

definizione:

```
double cubo(float c){...}
```

ritorno:

```
return (c*c*c);
```

Dichiarazione di una funzione

Sintassi di dichiarazione, detta prototipo (*prototyping*):

```
tipoRitorno nomeFunz(tipoPar1, ..., tipoParN) ;
```

Tipo ritornato dalla funzione, nome della funzione.

Tra parentesi: tipi dei parametri in ingresso separati da virgola.

Presuppone esista da qualche parte la definizione della funzione.

```
double cubo(float)
```

Tipo ritornato **double**, nome **cubo**, un parametro di tipo **float**.

Si potrebbero specificare anche i nomi dei parametri formali.

```
double cubo(float c) ;
```

E' una dichiar. valida. Il nome del parametro **c** però è superfluo.

Definizione di una funzione

Sintassi di definizione:

```
tipoRitorno nomeFunz(tipoPar1 par1,...,tipoParN parN)  
  
{      dichiarazioni di variabili;  
        istruzioni...  
        return (valore o espressione);  
  
}
```

- Tipo ritornato dalla funzione (se omesso viene considerato **int**)
- Nome della funzione.
- Tra parentesi tonde: tipo e nome dei parametri in ingresso – **parametri formali** – separati da virgola.

ATTENZIONE: Ciascun parametro deve essere preceduto dal suo tipo (il tipo va ripetuto anche se ci sono più parametri dello stesso tipo)

- Dichiarazioni di variabili (**variabili locali**)
- Blocco d'istruzioni che termina con **istruzione return**

Definizione di una funzione

```
double cubo(int c)
{
    return (c*c*c);
}
```

- Tipo ritornato: **double**, nome: **cubo**,
- tipo del primo e unico parametro formale: **float**,
- nome del primo e unico parametro: **c**.

return & chiamata di funzione

Il *valore di ritorno* è restituito con l'istruzione **return**.

```
return (espressione);    0    return espressione;
```

Una funzione è invocata mediante il nome e la lista di parametri conforme in tipo, numero e ordine alla lista dei parametri formali.

```
risultato = cubo(num) ;
```

La variabile **num** è il *parametro attuale*, è valido anche:

```
risultato = cubo(5) ;
```

Errori possibili: ordine, numero o discordanza di tipo .

Passaggio dei parametri

Il passaggio dei parametri avviene **per valore**: all'invocazione ogni parametro formale è inizializzato con il valore del parametro attuale.

Non sempre occorre la perfetta corrispondenza di tipo tra parametri formali e attuali, avvengono infatti *conversioni implicite di tipo*:

I parametri attuali **float** sono convertiti in **double** prima di essere passati alla funzione.

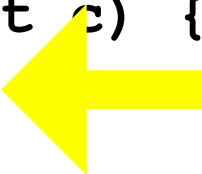
I parametri attuali **char** e **short int** sono convertiti in **int**.

(*Passaggio implicito*: definire una variabile globale sia alla funzione chiamante sia a quella chiamata.)

Definizione di una funzione (esempi)

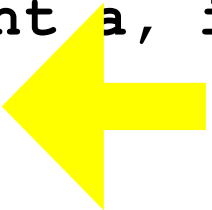
```
double cubo(float c) {  
    return (c*c*c);  
}
```

```
double cubo(float c) {  
    double risul;  
    risul=c*c*c;  
    return risul;  
}
```

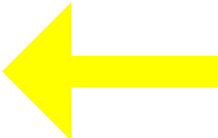


Definizione di una funzione (esempi)

```
int maxTest(int a, int b, int c) {  
    int max;  
    max=a;  
    if (b>max) max= b;  
    if (c>max) max= c;  
    return max;  
}
```



Definizione di una funzione (esempi)

```
int PrimoTest ( long num){  
    int primo = 1;   
    long d = 2;  
    long fine = (int) sqrt(num);  
    while ( d <= fine && primo==1) {  
        if ( num % d == 0 )  
            primo = 0;  
        d++;  
    }  
    return primo;  
}
```

Funzioni void

Per le funzioni che non restituiscono alcun valore, e funzioni che non hanno parametri si ha a disposizione un “tipo” speciale detto **void**.

```
void Menu()  
{  
    printf("\t Scrivi 1 per disegnare un quadrato pieno \n");  
    printf("\t Scrivi 2 per disegnare un quadrato vuoto \n");  
    printf("\t Scrivi 3 per disegnare due triangoli \n");  
    printf("\t Scrivi 0 per uscire \n\n");  
    return ;  
}
```